RAG Masternote

RAG

RAG is a technique that enhances a model's generation by retrieving the relevant information from external memory sources. An external memory source can be an internal database, a user's previous chat sessions, or the internet.

It was proposed for the knowledge-intensive tasks where all the available knowledge can not be input into the model directly as only information that is relevant to the query is retrieved and hand it over to the model, which will later reduce the hallucination bottleneck faced in the AI generation.

Context and RAG

Context Construction = Feature Engineering = giving the model the necessary information to process an input.

- The longer the context, the more likely the model is to focus on the wrong part of the context. RAG allows the model to use only the most relevant information for each query, reducing the number of input tokens while potentially increasing the model's performance.
- **Anthropic:** "If your knowledge base is smaller than 200000 tokens (about 500 pages of material), you can just include the entire knowledge base in the prompt that you give the model, with no need for RAG or similar methods."

RAG Architecture

- **Retriever:** retrieve information from external memory sources
- **Generator:** generate a response based on the retrieved information.

The success of the RAG system depends on the quality of its retriever.
and the retriever has two main functions: indexing and querying

- **Indexing:** involve processing data so that it can be quickly retrieved later.
    - How to index the data depends on how you want to retrieve it later on.
- **Querying:** Sending a query to retrieve the data.

Retrieval Algorithm

Retrieval has been around for a very long and is used for:

- Search engine
- Recommender systems
- Log analytics...

Note: Retrieval Vs Search
- **Retrieval** = limited to one database or system
- **Search** = involves retrieval across various systems

Retrieval works by ranking documents based on their relevance to a given query.
--> The retrieval algorithms differ based on how relevance scores are computed.

Two common retrieval mechanisms:

- **Term-based retrieval/lexical retrieval/Sparse retrieval:**
  - Find the relevant content with keywords instead of the dense numerical vector that captures the meaning.
    - **For example:** given the query "AI Engineering", the model will retrieve all the documents that contain "AI engineering".
    - **How it works**
      1. **Indexing:** Documents are broken into tokens (words), and an inverted index is built then mapping from each term to the list of documents containing it (position/frequencies)
      2. **Query matching:** When a query comes in, it is also tokenized, and the system looks up which documents contain those term.
      3. **Scoring:** Documents are ranked by relevance using a scoring function like the BM25 or the TF-IDF
    - **TF-IDF/BM25:**
      - **Term frequency (TF):** measures how often a query term appears in the document.
      - **Inverse Document Frequency (IDF):** rare item across the corpus are weighted higher than common ones (i.e., "the". "is" matter less than the "machine")
      - **Document length normalization:** prevents long documents from unfairly scoring higher just by containing more words.
  - **Characteristic:**
    - Sparse retriever represent the data using sparse vectors (the majority of the values are 0, the One-hot vector, a vector that is 0 everywhere except one value is 1). And the vector size is the length of the vocabulary. The value of 1 is in the index corresponding to the index of the term in the vocabulary
      - **For example**, if we have the dict {"food": 0, "book": 1, "apple":2}. then the One-hot vectors of them are $[1, 0, 0], [0, 1, 0], [0, 0, 1]$ respectively.
  - **Why it is used:**
    - **Exact match precision**
    - **No training is needed** (work out of the box, no embedding model required)
    - **Interpretable** as it is easy to see why a document matched
    - **Fast and Cheap**

        - **Robust to rare/unseen terms:** so the embedding model can struggle with out-of-vocabulary or highly technical/rare tokens that were not well represented in the training data
    - **However:**
        - **No semantic understanding:** "car" and "automobile" are treated as unrelated terms unless they literally match
        - **Sensitive to phrasing:** a synonym-heavy or paraphrased query might miss relevant documents entirely
        - **No conceptual reasoning:** cannot infer relevance from context or meaning, only surface-level word overlap.
- **Embedding-based retrieval/Semantic retrieval**:
    - Finds relevant documents based on meaning, not exact word overlap.
    - Aim to rank documents based on how closely their meanings align with the query.
        - With the embedding-based retrieval, indexing has an extra function: converting the original data chunks into embeddings. The database where the generated embeddings are stored is called a **vector database.**
        - Convert both the query and every document chunk into a dense vector (a list of numbers, typically 384-1536 dimensions) that captures its semantic meaning. Texts with similar meaning end up close together in this vector space even if they don't share a single word.
    - **How it works:**
        1. **Embedding**: use model like OpenAI's `text-embedding-3` ,... to convert text into a fixed-length vector
            - **For example:**
                - The dog is barking -> $[0.12, -0.34, 0.44, \dots]$
                - A canine is making noise -> $[0.14, -0.40, 0.40, \dots]$
                    - **Note:** These 2 sentences share zero keywords but land close together in the vector space, because the model learned the meaning during the training.
        2. **Indexing**: All document chunks are embedded ahead of timeand stored in a vector database (Pinecone, Weaviate, Chroma, FAISS, pgvector) often using approximate nearest neighbor (ANN) algorithms like HNSW for fast lookup at scale
        3. **Query and retrieval**: during the query time:
            - The query is embedded using the same model
            - The system computes similarity (Cosine Similarity or the dot product) between the query vector and all document vectors.
            - The top-k most similar chunks are returned.
    - **Why it is used:**
        - **Handle paraphrasing/synonyms**: "How do I reset my password" matches "steps to recover account access"

    - **Captures conceptual relevance**: Understand intent, not just surface words
    - **Cross-lingual potential**: Some multilingual embedding models can match a query in one language to documents in another.
    - **Good for vague/natural queries**: User rarely phrase the queries with the exact terminology used in source documents
- **However:**
    - **Weak on exact matches:** Struggles with product IDs, error codes, names, or rare technical terms that were not well-presented during embedding model training
    - **Semantic drifting:** Can retrieve topically-related but actually irrelevant content (two texts about "banks" confuse with the river vs finance context)
    - **Costly:** requires running a neutral network over every chunk (embedding cost) and often a GPU/ANN for faster index search
    - **Chunking sensitivity:** how we split the documents into chunks heavily affects retrieval quality because if it is too big then you dilute meaning, and if it is too small and you lose context
    - **Embedding model mismatch:** query and documents must be embedded with the same (or compatible) model, or the similarity score becomes meaningless
    - **Hard to interpret** because we cannot really explain why two vectors are close or have similar meaning...
- **Characteristic:**
    - A dense vector is a vector where the majority of the value are not 0.
    - **Note:** there is also sparse embedding, for example, **SPLADE (Sparse Lexical and Expansion)** it leverages embeddings generated by BERT but uses regularization to push most embedding values to 0
    - **Chunk size/overlap** is typically 200-800 tokens per chunk, often with some overlap to preserve context across boundaries
    - **Embedding model choice:** trade-off between dimensionality (accuracy) and speed/storage cost
    - **Similarity metric:** Cosine Similarity, but some models are trained specifically for dot-product retrieval
    - **ANN index type:** HNSW, IVF,.. and the trade off is search speed and recall accuracy

Table 6-2. Term-based retrieval and semantic retrieval by speed, performance, and cost.

	Term-based retrieval	Embedding-based retrieval
Querying speed	Much faster than embedding-based retrieval	Query embedding generation and vector search can be slow
Performance	Typically strong performance out of the box, but hard to improve Can retrieve wrong documents due to term ambiguity	Can outperform term-based retrieval with finetuning Allows for the use of more natural queries, as it focuses on semantics instead of terms
Cost	Much cheaper than embedding-based retrieval	Embedding, vector storage, and vector search solutions can be expensive

Term-based retrieval is generally much faster than embedding-based retrieval during both the indexing and query.

- **Hybrid Search** (Combining the embedding-based (dense/vector) retrieval with term-based (sparse/lexical) retrieval):
  - The embedding capture the semantic/conceptual similarity, whereas the term-based search catches exact keyword/entity matches, rare terms, acronyms, IDs,...
    - So combining both of them together tends to outperform either alone.
  - **How it works**
    1. **Run both retrievers in parallel**
       - **Sparse/term-based:** BM25 or TF-IDF over an inverted index, returns a ranked list with lexical relevance scores
       - **Dense/embedding-based:** Cosine/dot-product similarity search over a vector index (HNSW, IVF...) return a ranked list with similarity scores
    2. **Fuse the results**
       - **Reciprocal Rank Fusion (RRF):** ignore the raw score and focus entirely on rank position

$$\text{RRF}(d) = \sum_{i \in \{sparse, dense\}} \frac{1}{k + \text{rank}_i(d)}$$

Where the $rank_i(d)$ is the rank of the document d in retriever i's result list (1-indexed), and k is a smoothing constant (commonly $k = 60$)

       - **Weighted linear combination of normalized scores**: run both retrievers over the same candidate pool, normalize their scores onto a comparable scale, then blend them with a single turnable weight.
       - **Cascade/re-ranking**: no need to fuse two full retriever directly, use a cheap, high-recall first stage to narrow the field, then a more expensive, high-precision second stage to reorder just those candidate.

Comparing retrieval algorithm

The quality of a retriever can be evaluated based on the quality of the data it retrieves. Two metric often used by RAG evaluation frameworks are Context precision and Context

recall.
(Precision or Recall for short) + Context Precision = Context Relevance

- **Context precision:** Out of all the documents retrieved, what percentage is relevant to the query = of the chunks retrieved, how many were actually useful/relevant?
  - It measures signal-to-noise in the retrieved context. If the retriever retrieve 10 chunk and only 3 chunks are relevant to answering the question, meaning that the precision is LOW = Feeding the model a lot of junk
  - Many implementation like **(RAGAS)** go further by computing it as **"Rank-weighted"** metric, meaning that rewarding system that put the relevant chunks near the top of the retrieved list, not just anywhere in it. So retrieving 10 chunks where the relevance ones are ranked 1st and 2nd scores higher than the same 10 chunks with relevant ones ranked 8th and 9th.
- **Context recall:** Out of all the documents that are relevant to the query, what percentage is retrieved
  - It measures completeness/coverage. If the ground-truth answer requires 5 facts spread across your documents, and your retriever only surfaces 3 of them, then the recall is low even those 3 chunks were perfectly relevant (high precision)
  - Low recall = the retriever missing key important documents, causing the LLM to answer with incomplete information (or hallucinate to fill in the gap) even though it wrote a good answer with what it did get.

	HIGH PRECISION	**LOW PRECISION**
HIGH RECALL	Ideal: Got everthing needed, no noise	Got everything needed, but buried in junk
LOW RECALL	Clean but missed the key info	Worst case: Incomplete and noisy

Note: there is a tension between the two since retrieving more to boost the recall will end up risk adding noise that degrade the precision.

RAGAS to compute these two metrics automatically using an LLM-as-judge approach against reference answer.

- **Context Precision:** for each retrieve chunk, an LLM judge checks like "Was this chunk actually necessary or useful to arrive at the reference answer?" then it outputs a binary verdict (relevant/irrelevant) per chunk. Then it computes a "Rank weighted average precision"

$$\text{Precision@k} = \frac{\sum_{i=1}^{k} v_i}{k}$$

  where the Precision@K is the precision considering only the first k chunks.

The overall **Context Precision@K** is the rank-weighted average:

$$\text{Context Precision@K} = \frac{\sum_{k=1}^{K}(\text{Precision@k} \times v_k)}{\sum_{k=1}^{K} v_k}$$

This weighting means a relevant chunk near the top (small k) contributes to more of the Precision@k terms in the sum, so it boosts the score more than a relevant chunk buried near the bottom.

- **Context Recall:** the LLM judge takes the reference (answer), breaks it into individual claims/sentences, and check each one: "Can this claim be attributed to something in the retrieved context?"

$$\text{Context Recall} = \frac{\text{Claims in reference answer supported by the retrieved content}}{\text{Total claims in reference answer}}$$

To improve the Context Precision without affecting the Context Recall

- **Re-ranking:** retrieve wide (high recall pool, e.g., top 50), then use a cross-encoder re-ranker to pick the best top 5-10.
- **Metadata/filtering:** Pre-filter by document type, date, or section before semantic search, so irrelevant categories never enter the pool
- **Better chunking:** smaller, semantically coherent chunk reduce the chances a chunk is "half relevant or half noise
- **Similarity threshold:** drop chunks below a relevance score cutoff, but we have to be careful with the aggressive threshold as it may affect the context recall.

To improve the Context Recall without wrecking the Context Precision

- **Hybrid Search:** combine dense (embedding) search with sparse (BM25/keyword) search to catches cases where semantic search misses exact terms (IDs, jargon, names) or vice versa.
- **Query expansion/multi-query:** Generate several paraphrased versions of the user's query and retrieve for each, then merge
- **Larger K with re-ranking downstream:** Retrieve more candidates initially (top 30-50) rather than top 5, then let a reranker clean it up (same idea as above)
- **Better chunking overlay**: if the chunks are too small or isolated, relevant content can get split across two chunks and only one gets retrieved

Retrieval Optimization

Chunking Strategies

- **Fixed size chunking**: Splits text into equal sized chunks based on character or token count. Simple and fast but can cut sentences awkwardly.

- **Sentence based chunking**: Splits text at sentence boundaries. Keeps grammatical units intact.
- **Paragraph based chunking**: Uses paragraphs as natural chunk boundaries. Good for well structured documents.
- **Recursive chunking**: Tries splitting by larger units first (paragraphs), then falls back to smaller units (sentences, words) if chunks are still too big.
- **Sliding window chunking**: Creates overlapping chunks so context isn't lost at boundaries. Useful for maintaining continuity between chunks.
- **Semantic chunking**: Groups text based on meaning or topic similarity using embeddings. Produces more contextually coherent chunks.
- **Document structure based chunking**: Uses headings, sections, or markdown structure to define chunk boundaries. Works well for structured docs like manuals or reports.
- **Agentic or LLM based chunking**: Uses an LLM to decide where meaningful breaks should occur based on content understanding. More expensive but highly context aware.
- **Token based chunking**: Splits based on token limits of the embedding or LLM model to avoid truncation issues.
- **Hybrid chunking**: Combines multiple strategies, for example structure based splitting plus semantic refinement, to balance speed and quality.

Reranking

It is useful when we need to reduct the number of retrieved documents, either to fit them into your model's context window or to reduce the number of input token.

- **Cross encoder reranking**: Passes query and document together through a transformer model to compute a relevance score. Highly accurate but slower since it processes pairs one at a time.
- **Bi encoder reranking**: Encodes query and documents separately into embeddings, then compares via cosine similarity. Faster than cross encoders but less precise.
- **LLM based reranking**: Uses a large language model to judge and reorder retrieved documents based on relevance to the query. Flexible and context aware but costly at scale.
- **Pointwise reranking**: Scores each document independently against the query, then sorts by score.
- **Pairwise reranking**: Compares documents two at a time to decide which is more relevant, building an order through repeated comparisons.
- **Listwise reranking**: Considers the entire list of retrieved documents together and outputs an optimal ranking in one pass.
- **Reciprocal rank fusion**: Combines rankings from multiple retrieval methods, for example keyword search and vector search, by summing reciprocal rank scores.
- **Metadata based reranking**: Boosts or penalizes documents using metadata signals like recency, source authority, or popularity.

- **Diversity aware reranking (MMR)**: Uses Maximal Marginal Relevance to balance relevance with diversity, reducing redundant or overly similar results.
- **Hybrid reranking**: Combines multiple signals, for example semantic similarity plus keyword match plus metadata, into a single reranking score.

Query mod

- **Query rewriting:** Reformulate the original query into a clearer or more effective version for retrieval, often using an LLM to proceed this process.
- **Query expansion**: Adds synonyms, related terms, or context to the query to capture more relevant documents.
- **Query decomposition**: Breaks a complex query into smaller sub questions that can be answered separately then combined.
- **Multi query generation**: Generates several variations of the same query to retrieve a broader and more diverse set of documents.
- **Hypothetical document embeddings (HyDE)**: Generates a hypothetical answer to the query first, then uses that answer's embedding to retrieve similar real documents.
- **Step back prompting**: Asks a more general or abstract version of the query first to retrieve broader context before answering the specific question.
- **Query routing**: Analyzes the query intent and directs it to the most appropriate retrieval source, index, or tool.
- **Contextual query rewriting**: Incorporates conversation history or prior turns to rewrite a query that depends on earlier context, common in chat based RAG.
- **Query normalization**: Cleans and standardizes the query, for example fixing typos, casing, or removing filler words, before retrieval.
- **Pseudo relevance feedback**: Uses initial retrieval results to refine and reissue an improved query automatically.

Contextual Retrieval

Used it to augment each chunk with relevant context to make it easier to retrieve the relevant chunks.

- **Contextual embeddings**: Adds surrounding document or chunk context to each piece of text before embedding it, so the embedding captures meaning beyond just the isolated chunk.
- **Contextual BM25**: Combines traditional keyword based search with added context per chunk, improving lexical matching for specific terms or names.
- **Chunk context annotation**: Prepends a short LLM generated summary of where a chunk fits within the larger document, for example section title or surrounding topic, before indexing.
- **Hybrid contextual retrieval**: Combines contextual embeddings with contextual BM25 and often reranking, to capture both semantic meaning and exact keyword matches.

- **Conversation history context**: Incorporates prior turns of a conversation into the retrieval query, so follow up questions retrieve relevant results even without repeating context.
- **Document level context injection**: Adds metadata like document title, author, date, or section headers into the chunk representation to preserve context lost during chunking.
- **Whole document context caching**: Uses long context models to reference the entire source document during generation, reducing retrieval errors caused by fragmented chunks.
- **Query context enrichment**: Expands the user query with relevant background information or prior context before running retrieval.
- **Entity and relationship context**: Uses knowledge graphs or extracted entities to preserve relationships between concepts across chunks, improving retrieval for connected information.
- **Feedback loop context**: Uses previous retrieval results or user interactions to refine what context gets pulled in future queries.

Multimodal RAG

If your generator is manipulated, its context might be augmented not only with text documents but also with images, video, audio, etc... from the external database

If you want to retrieve images based on their content., you will need to have a way to compare images to queries. If queries are texts, you will need a multimodal embedding model that can generate embeddings for both images and texts.
For example: Uses models like CLIP
- Generate CLIP embeddings for all your data, both texts and images, and store them in the vector database
- Given a query, generate its CLIP embedding
- Query in the vector database for all images and texts whose embeddings are close to the query embedding

RAG with tabular data

Most applications work not only with unstructured data like texts and images but also with tabular data.

RAG with tabular data is tricky because tables have structure (rows, columns, relationships) that gets lost if you treat them like plain text.

It is hard for RAG because the tables rely on row and column context to mean anything, for example a number only makes sense with its column header and row label. Naive chunking that splits a table into raw text fragments destroys these relationships and makes retrieval nearly useless.

So we have to use these strategies to save the model from wrecking

- **Row level chunking**: Converts each row into a natural language sentence or key value pairs, including the column headers as context, then embeds and indexes each row separately.
- **Table to text conversion**: Uses an LLM to summarize or describe the table content in natural language before embedding, so semantic search can match against descriptive text.
- **Table summarization**: Generates a short summary of what the table contains, for example its topic and columns, and indexes the summary alongside or instead of raw data for retrieval.
- **Schema aware retrieval**: Stores table metadata like column names and data types separately, so queries can be matched to the right table structure before pulling specific rows.
- **SQL or structured query generation**: Uses an LLM to translate the natural language question into a SQL query (Text to SQL), then runs it directly against a database instead of relying purely on vector search.
- **Hybrid text and table indexing**: Keeps tables in a structured store, for example a database or dataframe, while indexing surrounding narrative text normally, then merges both types of context at query time.
- **Cell level embedding with context**: Embeds individual cells along with their row and column headers to preserve meaning, useful for very large or sparse tables.
- **Multi vector retrieval**: Stores multiple representations of the same table, for example a summary, row level chunks, and raw structured data, then chooses the best one depending on the query type.
- **Table QA models**: Uses specialized table question answering models that can reason directly over structured table input rather than converting everything to text first.
- **Layout aware extraction**: For tables inside PDFs or scanned documents, uses layout detection tools to correctly extract rows and columns before any indexing happens, since OCR alone often breaks table structure.